

FORMATION EMBARQUÉE

# TP4 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

# TP 4 — Fiche de séance 2 : table d'échange Modbus (2 h)

## En-tête pédagogique

|                  |                                                                                                                                      |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectifs</b> | Concevoir une table d'échange documentée ; distinguer zones lecture/écriture ; implémenter un mot de vie ; borner les entrées réseau |
| <b>Prérequis</b> | Séance 1 : régulation cuve fonctionnelle                                                                                             |
| <b>Outils</b>    | Machine Expert Basic + simulateur                                                                                                    |
| <b>Livrable</b>  | Document de table + section « Échange » dans le programme                                                                            |

## Déroulé minuté

### 0:00-0:40 — Le document de table (livrable clef)

Une table Modbus est un **document contractuel** : Modbus ne transporte que des mots 16 bits, tout le *sens* est dans ce document. Sans lui, le client PC ne peut rien décoder. Rédige-le (reprends le format du TD 08 ex. 3) :

| Mot    | Adr. | Sens     | Contenu                                                          | Codage                    |
|--------|------|----------|------------------------------------------------------------------|---------------------------|
| %MW100 | 100  | API → PC | État : b0=P1, b1=P2, b2=vanne, b3=nivTH, b4=nivTB, b5=defCapteur | bits                      |
| %MW101 | 101  | API → PC | Niveau                                                           | pour-mille (0-1000)       |
| %MW102 | 102  | API → PC | Heures P1                                                        | h ×10                     |
| %MW103 | 103  | API → PC | Heures P2                                                        | h ×10                     |
| %MW104 | 104  | API → PC | <b>Mot de vie</b>                                                | +1/s, boucle 0-65535      |
| %MW110 | 110  | PC → API | Commande : b0=autoriser, b1=acquit                               | bits                      |
| %MW111 | 111  | PC → API | Consigne haute                                                   | pour-mille, borné 500-800 |

Règles à énoncer : - **Zones disjointes** : jamais les deux côtés écrivains du même mot (API → PC et PC → API séparés). Sinon, écrasement mutuel. - **Entiers ×10** plutôt que Real : un flottant occupe 2 mots avec un ordre (word swap) dépendant de l'équipement — source n°1 de bugs Modbus. - Le **mot de vie** : indispensable. Une connexion TCP ouverte ne prouve pas que le programme tourne (la CPU peut être en STOP, figée). Le PC surveille que ce compteur bouge.

### 0:40-1:20 — Section « Échange » dans le programme

Sur M221, les **%MW** sont nativement lisibles/écrivables en Modbus TCP (holding registers, adresse = numéro du mot). Rien à « publier », mais il faut **recopier** explicitement les états dans la zone API → PC et

lire + borner la zone PC → API :

```
// Recopie des états vers %MW100 (bloc opération, bit à bit)
%MW100 := 0
Si CMD_P1      alors %MW100 := %MW100 OU 1      (b0)
Si CMD_P2      alors %MW100 := %MW100 OU 2      (b1)
Si VANNE       alors %MW100 := %MW100 OU 4      (b2)
Si %MW10>900   alors %MW100 := %MW100 OU 8      (b3)
Si %MW10<50    alors %MW100 := %MW100 OU 16     (b4)
Si DEF_CAPTEUR alors %MW100 := %MW100 OU 32     (b5)

%MW101 := %MW10      // niveau
%MW102 := %MW30 / 360 // secondes → dixièmes d'heure, selon ton codage
```

## 1:20-1:45 — Mot de vie et bornage

**Mot de vie** : incrémenter `%MW104` chaque seconde (bit système `%S6` qui bascule à 1 Hz + détection de front + bloc opération). Laisser déborder naturellement (0-65535).

**Bornage de la consigne** : l'automate ne fait JAMAIS confiance au réseau.

```
Si %MW111 < 500 alors %MW111 := 500
Si %MW111 > 800 alors %MW111 := 800
// puis seulement, utiliser %MW111 comme seuil haut de régulation
```

Un PC (ou un pirate) qui écrit 9999 dans `%MW111` ne doit pas faire déborder la cuve.

✅ **Point de contrôle** : au simulateur, vérifie que `%MW100` reflète bien les états (allume une pompe → b0 passe à 1), que `%MW104` s'incrémente, et qu'écrire 9999 dans `%MW111` le ramène à 800.

## 1:45-2:00 — Commit + doc

Range le document de table dans le dépôt ( `docs/table_modbus.md` ) — c'est le livrable que tu donneras au développeur du superviseur (toi-même en séance 3).

## Erreurs fréquentes

| Symptôme                           | Cause                              | Remède                                 |
|------------------------------------|------------------------------------|----------------------------------------|
| Le PC lit des valeurs incohérentes | pas de recopie état → %MW100       | ajouter la section Échange             |
| Consigne 9999 appliquée            | pas de bornage côté API            | borner à chaque cycle                  |
| PC ne détecte pas la panne CPU     | mot de vie absent ou figé côté API | %S6 + front + incrément                |
| b0/b1 mélangés                     | masques de bits erronés            | vérifier les puissances de 2           |
| Le PC écrit une %Q directement     | zone d'écriture mal conçue         | le PC écrit des DEMANDES, l'API valide |

## Travail à la maison (30 min)

Complète la table avec 3 mots de réserve (%MW105-107, mis à 0) pour les extensions futures — un réflexe pro : on prévoit toujours de la place. Et documente : que doit faire le PC s'il lit %MW104 identique 3 fois de suite ? (*afficher une alarme « automate figé » — implémenté séance 3*).

➡ Fiche suivante : [Séance 3 — Client de supervision PC](#)